

Peer-to-Peer File Distribution

1 Introduction

Most traditional Internet applications (**Web**, **E-mail**, **DNS**) rely on a **client-server model**, where servers are centralized and always on.

In contrast, a **Peer-to-Peer (P2P) architecture**:

- Minimizes reliance on centralized servers.
- Uses intermittently connected **peers** (PCs, laptops, smartphones).
- Peers are user-controlled, not owned by providers.

Key Idea

File distribution is a natural P2P application: A large file (e.g., Linux ISO, software update, video) is distributed not only by the server but also by peers who help redistribute chunks.

2 Scalability of P2P Architectures

2.1 System Model

We distribute a file of size F bits to N peers:

- Server upload rate: u_s .
- Peer i upload rate: u_i , download rate: d_i .
- Minimum download rate: $d_{min} = \min(d_1, d_2, \dots, d_N)$.

2.2 Client-Server Distribution Time

$$D_{CS} = \max\left(\frac{NF}{u_s}, \frac{F}{d_{min}}\right)$$

Observation

- Server must send all NF bits.
- Slowest peer requires F/d_{min} seconds.
- **Scales poorly:** As N increases, D_{CS} grows linearly.

2.3 P2P Distribution Time

$$D_{P2P} = \max \left(\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i} \right)$$

Scalability Result

- Server uploads the file only once.
- Peers contribute upload bandwidth $\Rightarrow$ system becomes self-scalable.
- P2P handles large-scale distribution efficiently.

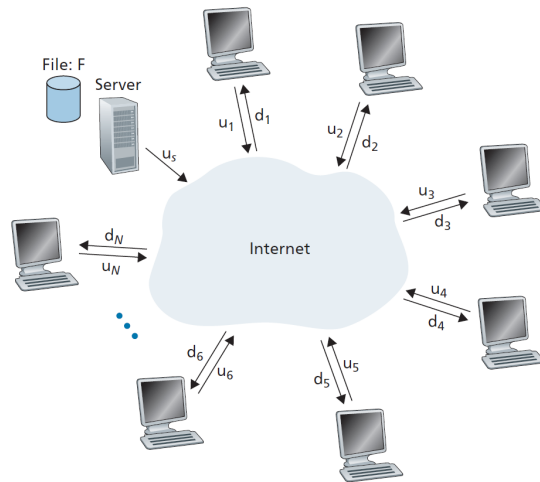


Figure 1: Distribution time: Client-Server vs P2P.

3 BitTorrent Protocol

3.1 Torrent Basics

- A **torrent** = collection of peers sharing a file.

- File is divided into fixed-size chunks (e.g., 256 KB).
- Each peer both **downloads** and **uploads** chunks.
- Peers can leave and rejoin freely.

3.2 Tracker

- Central node coordinating peer participation.
- Maintains active peer list.
- Provides each new peer with a random subset of neighbors.

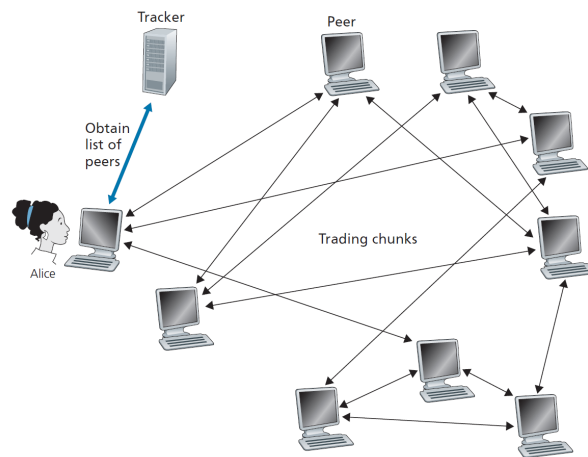


Figure 2: BitTorrent distribution with tracker and peers.

3.3 Peer Interaction

Steps for a joining peer (Alice):

1. Tracker provides peer list.
2. Alice establishes TCP connections with neighbors.
3. Requests chunks via **rarest-first**.
4. Uploads based on **tit-for-tat**.

3.3.1 Rarest-First Strategy:

- Prioritize downloading rarest chunks.
- Prevents bottlenecks due to missing chunks.

3.3.2 Tit-for-Tat Incentive:

- Upload to top 4 peers giving best download rates.
- Re-evaluate every 10s.
- Every 30s: optimistically unchoke a random peer.
- Ensures fairness and discourages free-riding.

3.4 Other Features

- Random-first piece strategy (bootstrap).
- Endgame mode for remaining chunks.
- Pipelining requests.
- Anti-snubbing against non-cooperative peers.

Why BitTorrent Works

- Self-scalable: peers are both consumers and redistributors.
- Tit-for-tat prevents free-riding.
- Rarest-first ensures balanced chunk availability.
- Widely adopted: millions of torrents active.

4 Distributed Hash Tables (DHT)

- P2P networks can implement distributed databases.
- Store key-value records across peers.
- Used in BitTorrent for peer discovery (**trackerless torrents**).

Key Takeaways

Summary

- **Client-Server:** poor scalability, server bottleneck.
- **P2P:** self-scalable; more peers $\Rightarrow$ more capacity.
- **BitTorrent:** tracker, rarest-first, and tit-for-tat ensure efficient distribution.
- **Beyond File Sharing:** P2P concepts extend to DHT-based distributed systems.